# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA4305**

---

**COURSE OUTCOME COVERED**

**CO 3:** Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

---

**QUESTIONS:** **Total Marks: 30**

## Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

---

## Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

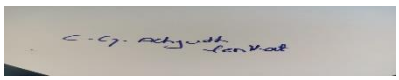
---

## Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

***Code of Conduct:***

*I C G Achyuth Venkat 192311102 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:*** 

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage (%) | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Understanding of Scenario | 20% | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps | Partial understanding; misses key aspects | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20% | Accurately identifies all relevant issues and factors involved | Identifies most key issues with minor omissions | Identifies limited issues; some irrelevant points | Unable to identify key issues |
| Application of Concepts | 25% | Applies appropriate concepts effectively to address the scenario | Applies concepts with minor errors | Limited or partially correct application | Unable to apply relevant concepts |
| Decision Making | 20% | Makes well-reasoned, logical decisions with strong rationale | Makes reasonable decisions with some justification | Makes weak decisions with limited justification | No clear decisions made or rationale provided |
| Clarity of Response | 15% | Response is clear, structured, and logically presented | Generally clear with minor issues | Somewhat unclear or poorly structured | Disorganized and difficult to understand |

**Question 1: Servlet-Based Student Registration System**

**1. Architecture of a Servlet-Based Application**

A servlet-based web application generally follows this architecture:

**Client → Web Server → Servlet Container → Servlet → Database → Response**

- **Client:** Browser sends HTTP request.
- **Web Server:** Receives the request.
- **Servlet Container:** Manages servlet execution and lifecycle.
- **Servlet:** Processes GET/POST requests and business logic.
- **Database:** Stores student information.

- **Response:** Servlet sends HTML/JSON response back to the browser.

**Basic Structure**

Browser
↓
HTTP Request
↓
Web Server
↓
Servlet Container (Tomcat)
↓
StudentServlet
↓
Database
↓
HTTP Response
↓
Browser

**2. HTML Registration Form**

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Registration</title>
</head>
<body>

<h2>Student Registration</h2>

<form action="student" method="post">

   <label>Name:</label>
   <input type="text" name="name" required><br><br>

   <label>Email:</label>
   <input type="email" name="email" required><br><br>

   <label>Course:</label>
   <input type="text" name="course" required><br><br>

   <button type="submit">Register</button>
</form>
</body>
</html>
```

**3. Servlet Implementation Using GET and POST**

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/student")
public class StudentServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<h2>Student Details - GET</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");
```

```
        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }
}
```

**4. Servlet Life Cycle**

**init()**

- Called only once when the servlet is created.
- Used for initialization tasks such as loading configuration.

**service()**

- Called for every client request.
- Determines whether the request is GET, POST, etc.
- Internally calls doGet(), doPost(), etc.

**destroy()**

- Called once before the servlet is removed.
- Used to release resources.

```
Servlet Created
      ↓
    init()
      ↓
   service()
      ↓
 doGet()/doPost()
      ↓
   service()
      ↓
   destroy()
```

**5. Appropriate Method for Sensitive Data**

**POST is more appropriate** for sensitive data because form data is sent in the request body instead of being appended to the URL.

However, **POST alone does not encrypt data**. Sensitive information should always be transmitted using **HTTPS**, with proper server-side validation and secure authentication.

---

**Question 2: Session Management Using HttpSession and Cookies**

**1. Session Management Architecture**

```
Login Page
    ↓
Login Servlet
    ↓
Authentication
    ↓
```

HttpSession Created
↓
Session ID
↓
Cookie (JSESSIONID)
↓
Subsequent Requests
↓
Server Identifies User

**2. Login Servlet Using HttpSession**

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if ("admin".equals(username) && "1234".equals(password)) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            response.sendRedirect("home");
        } else {
            response.getWriter().println("Invalid username or password");
        }
    }
}
```

**3. Accessing the Session**

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
```

```
import jakarta.servlet.http.*;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            response.getWriter().println(
                "Welcome " + username
            );

        } else {
            response.sendRedirect("login.html");
        }
    }
}
```

**4. Session Tracking Using Cookies**

Servlet containers normally create a session ID such as:

JSESSIONID=ABC123XYZ

The browser stores this identifier in a cookie and sends it with subsequent requests.

The server uses the session ID to find the corresponding HttpSession.

Browser

↓

JSESSIONID

↓

Server

↓

HttpSession

↓

User Information

**5. Creating a Cookie Manually**

```
Cookie cookie = new Cookie("username", username);
cookie.setMaxAge(3600);
```

```
response.addCookie(cookie);
```
Reading the cookie:
```
Cookie[] cookies = request.getCookies();

if (cookies != null) {
    for (Cookie cookie : cookies) {
        if ("username".equals(cookie.getName())) {
            System.out.println(cookie.getValue());
        }
    }
}
```

**6. Session vs Cookies**

| HttpSession | Cookies |
|---|---|
| Data is primarily maintained on server | Data is stored in browser |
| Session ID identifies the user | Cookie contains stored data/identifier |
| Better for server-side user state | Useful for preferences and identifiers |
| Can store larger server-side objects | Limited storage size |
| Usually uses JSESSIONID cookie | Uses application-defined cookies |

**Conclusion:** HttpSession is preferred for maintaining authenticated user sessions, while cookies commonly help the browser carry the session identifier.

**Question 3: JDBC-Based Product Management**

**1. JDBC Architecture**

JDBC stands for **Java Database Connectivity**.

Java Application
↓
JDBC API
↓
JDBC Driver
↓
Database
↓
SQL Query
↓
ResultSet
↓
Java Application

Main components:

- DriverManager
- Connection
- Statement / PreparedStatement

- ResultSet
- SQLException

**2. Database Table**

Example MySQL table:

```
CREATE DATABASE ecommerce;

USE ecommerce;

CREATE TABLE product (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   price DOUBLE,
   quantity INT
);
```

**3. JDBC Application**

```
import java.sql.*;

public class ProductJDBC {

   static final String URL =
      "jdbc:mysql://localhost:3306/ecommerce";

   static final String USER = "root";
   static final String PASSWORD = "root";

   public static void main(String[] args) {

      insertProduct("Laptop", 55000, 10);
      insertProduct("Keyboard", 1500, 20);

      retrieveProducts();
   }

   static void insertProduct(String name,
                             double price,
                             int quantity) {

      String sql =
         "INSERT INTO product(name, price, quantity) VALUES (?, ?, ?)";

      try (Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);
```

```
         PreparedStatement ps =
             con.prepareStatement(sql)) {

        ps.setString(1, name);
        ps.setDouble(2, price);
        ps.setInt(3, quantity);

        ps.executeUpdate();

        System.out.println("Product inserted successfully.");

    } catch (SQLException e) {

        System.out.println(
            "Database error: " + e.getMessage()
        );
    }
}

static void retrieveProducts() {

    String sql = "SELECT * FROM product";

    try (Connection con =
             DriverManager.getConnection(URL, USER, PASSWORD);

         PreparedStatement ps =
             con.prepareStatement(sql);

         ResultSet rs =
             ps.executeQuery()) {

        while (rs.next()) {

            System.out.println(
                "ID: " + rs.getInt("id")
            );

            System.out.println(
                "Name: " + rs.getString("name")
            );

            System.out.println(
```

```
                "Price: " + rs.getDouble("price")
            );

            System.out.println(
                "Quantity: " + rs.getInt("quantity")
            );

            System.out.println("----------------");
        }

    } catch (SQLException e) {

        System.out.println(
            "Database error: " + e.getMessage()
        );
    }
  }
}
```

**4. Steps Involved in JDBC Connectivity**

**Step 1: Import JDBC packages**

```
import java.sql.*;
```

**Step 2: Establish connection**

```
Connection con =
   DriverManager.getConnection(URL, USER, PASSWORD);
```

**Step 3: Create PreparedStatement**

```
PreparedStatement ps =
   con.prepareStatement(sql);
```

**Step 4: Set values**

```
ps.setString(1, name);
ps.setDouble(2, price);
ps.setInt(3, quantity);
```

**Step 5: Execute SQL**

For INSERT:

```
ps.executeUpdate();
```

For SELECT:

```
ResultSet rs = ps.executeQuery();
```

**Step 6: Process ResultSet**

```
while(rs.next()) {
   System.out.println(rs.getString("name"));
}
```

**Step 7: Close resources**

Use **try-with-resources** so connections, statements, and result sets are automatically closed.

**5. Exception Handling**

JDBC operations can throw SQLException.

```
try {
    // JDBC operations
}
catch(SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
```

Using PreparedStatement is recommended because it safely handles parameters and helps prevent **SQL injection**.